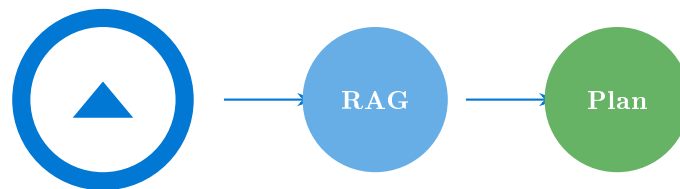


Project Submission Report

AI Travel Concierge Agent

Semantic Kernel Agentic Architecture with RAG



Student Name: Rodolfo Lerma

Date: February 2, 2026

Project: Udacity AI Foundry Project 3

Repository: <https://github.com/rodolfolemacontreras/-Udacity-Foundry-Project3>

Contents

1	Architecture Diagram	2
1.1	Component Summary	2
2	Azure Resources	3
2.1	Azure OpenAI Deployments	3
2.2	Azure Cosmos DB	3
2.3	Azure AI Foundry Agent with Bing Grounding	5
3	Test Results	7
3.1	Unit Tests	7
3.2	System Health Check	7
4	LLM-as-Judge Evaluation	10
4.1	Evaluation Criteria and Weights	10
4.2	Evaluation Results	10
5	RAG Implementation	12
5.1	Knowledge Base	12
5.2	Embedding Configuration	12
5.3	RAG Ingestion Results	13
6	Key Implementation Details	14
6.1	8-Phase State Machine	14
6.2	Tool Integration	14
6.3	Pydantic Output Validation	14
6.4	Bing Grounding Integration	14
7	Conclusions	15
7.1	Technology Stack Summary	15
7.2	Repository	15

1 Architecture Diagram

The following diagram illustrates the AI Travel Concierge Agent architecture, showing the 8-phase state machine workflow with Semantic Kernel orchestration, tool integrations, and RAG-enhanced knowledge retrieval.

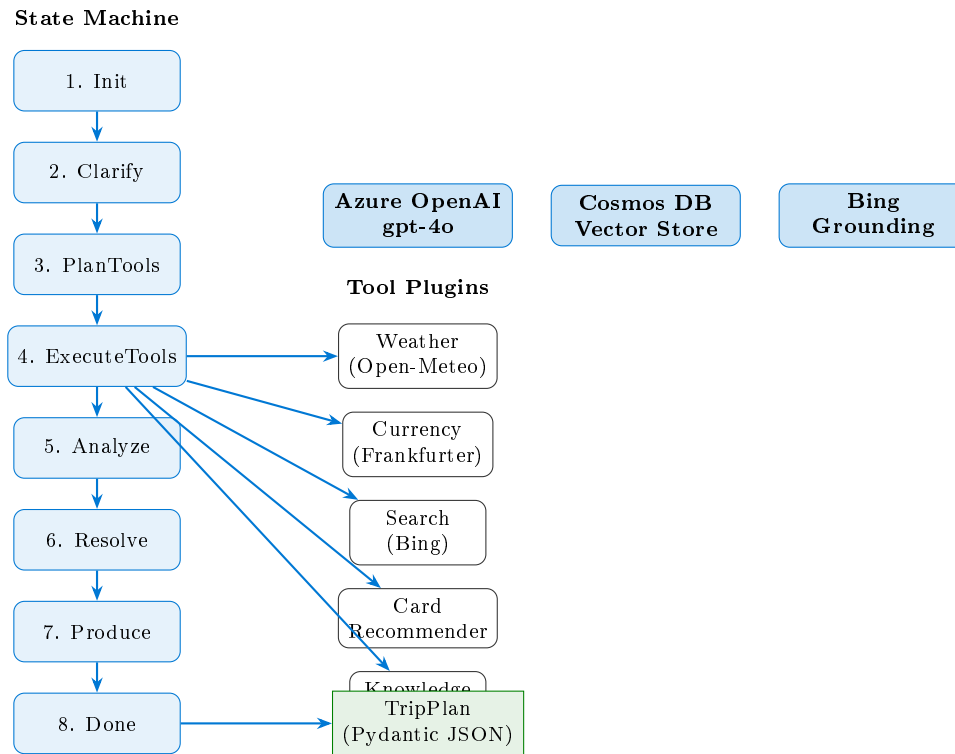


Figure 1: AI Travel Concierge Agent Architecture

1.1 Component Summary

Table 1: System Components

Component	Technology	Purpose
Orchestration LLM	Semantic Kernel 1.36.1 Azure OpenAI gpt-4o	Agent workflow and tool coordination Natural language understanding and generation
Embeddings	text-embedding-3-small	Vector embeddings for RAG retrieval
Vector Store	Azure Cosmos DB	Knowledge storage with vector search
Web Search	Bing Grounding (AI Foundry)	Real-time travel information retrieval
Validation	Pydantic 2.11.7	Structured output validation (TripPlan)

2 Azure Resources

2.1 Azure OpenAI Deployments

The following models are deployed in the Azure AI Services resource:

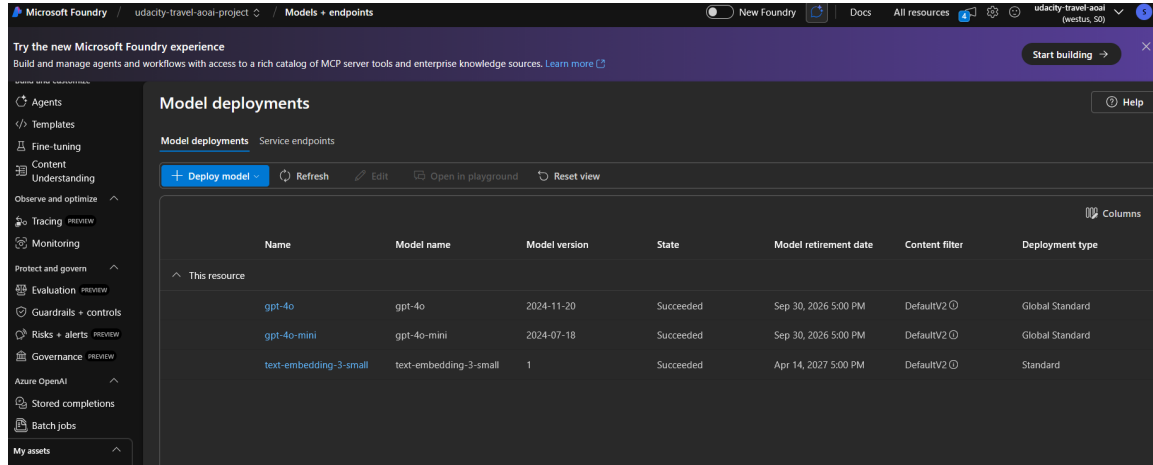


Figure 2: Azure OpenAI Model Deployments showing gpt-4o, gpt-4o-mini, and text-embedding-3-small

Table 2: Model Deployment Configuration

Deployment	Model	Type	TPM
gpt-4o	gpt-4o	GlobalStandard	10,000
gpt-4o-mini	gpt-4o-mini	GlobalStandard	10,000
text-embedding-3-small	text-embedding-3-small	Standard	10,000

2.2 Azure Cosmos DB

Vector-enabled Cosmos DB stores knowledge snippets with embeddings for RAG retrieval:

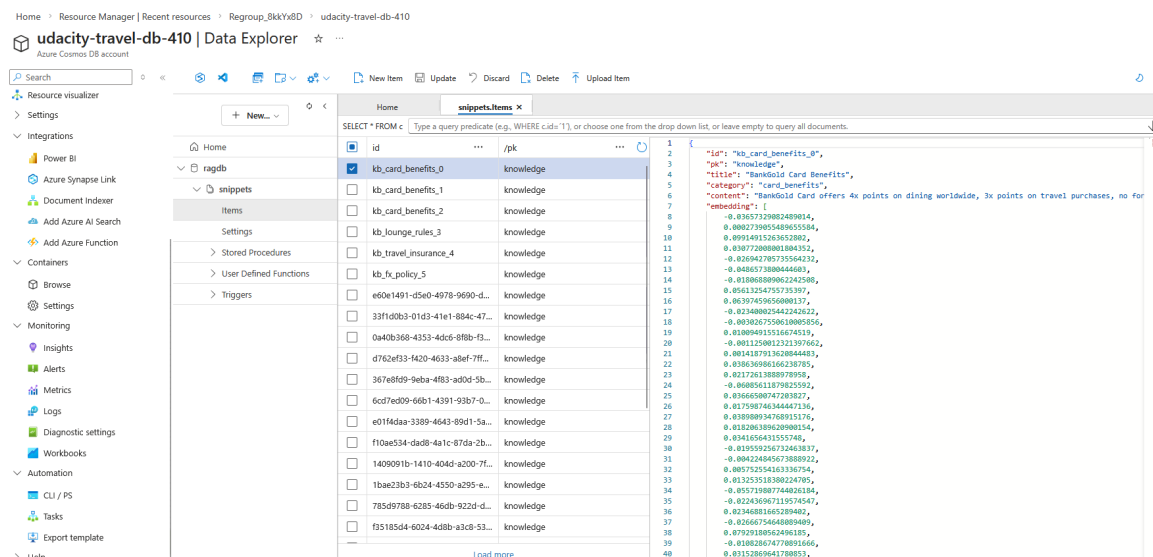


Figure 3: Cosmos DB Data Explorer showing knowledge snippets with embedding vectors

Table 3: Cosmos DB Configuration

Setting	Value
Account Name	udacity-travel-db-410
Database	ragdb
Container	snippets
Partition Key	/pk
Capacity Mode	Serverless
Region	West US

2.3 Azure AI Foundry Agent with Bing Grounding

The Travel Concierge Agent uses Azure AI Foundry with Bing Grounding for real-time web search:

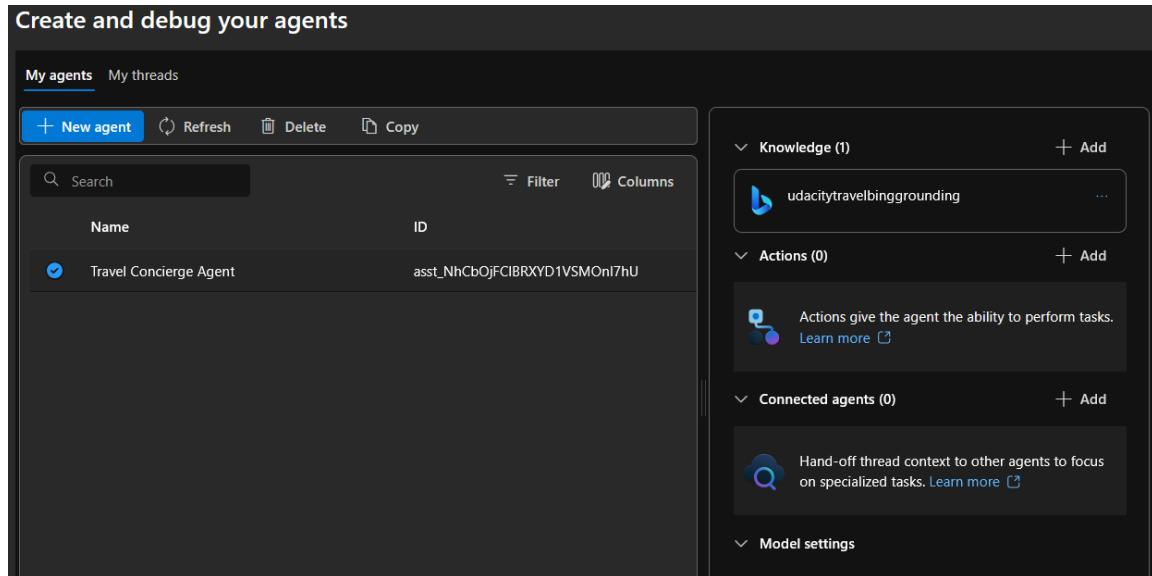


Figure 4: Azure AI Foundry Agent with Bing Search tool connected

Table 4: AI Foundry Agent Configuration

Setting	Value
Project Name	udacity-travel-aoai-project
Agent Name	Travel Concierge Agent
Agent ID	asst_NhCbOjFCIBRXYD1VSMOnI7hU
Model	gpt-4o
Tool	Bing Grounding (Search)

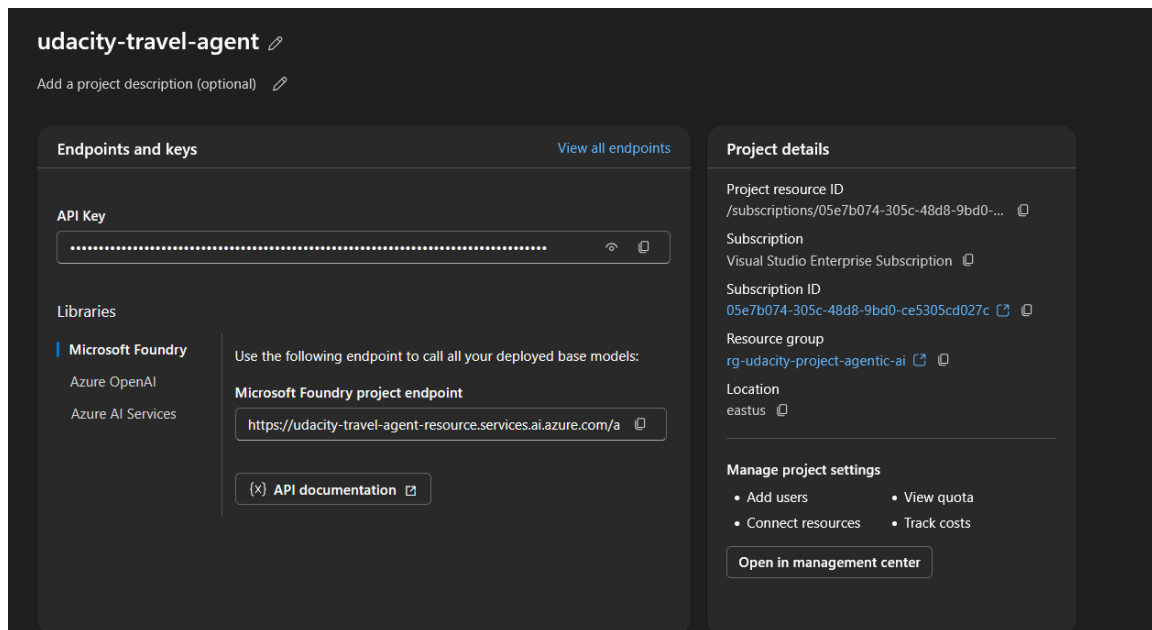


Figure 5: Azure AI Foundry Project Overview

3 Test Results

3.1 Unit Tests

All 76 unit tests pass successfully:

Table 5: Unit Test Results by Module

Test Module	Tests	Status
test_models.py	12	PASS
test_state.py	24	PASS
test_memory.py	18	PASS
test_memory_integration.py	14	PASS
test_tools.py	8	PASS
Total	76	All Passing

3.2 System Health Check

The system check validates all components are properly configured:

```
(.venv) PS C:\Training\Udacity\Microsoft_Foundry\Projects\Project_3> $env:PYTHONPATH = "."; python app/scripts/system_check.py
Travel Concierge Agent - System Health Check
=====
Timestamp: 2026-02-02 08:54:11

[RUNNING] Environment Variables...

[CHECK] Environment Variables
-----
[OK] AZURE_OPENAI_ENDPOINT
[OK] AZURE_OPENAI_API_VERSION
[OK] AZURE_OPENAI_CHAT_DEPLOYMENT
[OK] AZURE_OPENAI_EMBED_DEPLOYMENT
[OK] AZURE_OPENAI_KEY
[OK] COSMOS_ENDPOINT
[OK] COSMOS_KEY
[OK] COSMOS_DB
[OK] COSMOS_CONTAINER
[OK] COSMOS_PARTITION_KEY

[RUNNING] Azure OpenAI...

[CHECK] Azure OpenAI Service
-----
[OK] Chat: pong
[OK] Embedding: 1536 dimensions

[RUNNING] Cosmos DB...

[CHECK] Cosmos DB Service
-----
[OK] Cosmos DB connected (found 1 item(s))
```

Figure 6: System Health Check Results (Part 1)


```
[RUNNING] Tools...

[CHECK] Tool Integration
-----
[OK] Weather: N/A
[OK] FX: 100 USD = 84.46 EUR
[OK] Card: N/A
[OK] Knowledge: Found 5 results
[OK] Search: 1 result(s)
[OK] All tools working

[RUNNING] Grounding Search...

[CHECK] Azure Grounding Search
-----
Querying Agent for: "best restaurants in Tokyo 2025"
[OK] Received 1 result(s):

1. Web search: best restaurants in Tokyo 2025
   URL: https://bing.com
   Here are some standout restaurants in Tokyo as of 2025:

1. **SÉZANNE** - Locate...

[RUNNING] State Management...

[CHECK] State Management
-----
[OK] State management: Working

[RUNNING] Memory Systems...

[CHECK] Memory Systems
-----
[OK] Short and long-term memory: Working

[RUNNING] Knowledge Base...

[CHECK] Knowledge Base
-----
[OK] Knowledge base: Found 1 BankGold dining benefits
```

Figure 7: System Health Check Results (Part 2)

```

[RUNNING] Knowledge Base...

[CHECK] Knowledge Base
-----
[OK] Knowledge base: Found 1 BankGold dining benefits

Health Check Summary
=====
[PASS] Environment Variables
[PASS] Azure OpenAI
[PASS] Cosmos DB
[PASS] Tools
[PASS] Grounding Search
[PASS] State Management
[PASS] Memory Systems
[PASS] Knowledge Base

Overall: 8/8 checks passed

```

Figure 8: System Health Check Results (Part 3)

Table 6: System Check Results Summary

Check	Status
Environment Variables	PASS
Azure OpenAI Connection	PASS
Cosmos DB Connection	PASS
Tools (Weather, FX, Search, Card, Knowledge)	PASS
Grounding Search (Bing via AI Foundry)	PASS
State Management (8-phase workflow)	PASS
Memory Systems (Short-term + Long-term)	PASS
Knowledge Base (Card benefits retrieval)	PASS
Total	8/8 Passed

4 LLM-as-Judge Evaluation

The agent was evaluated using an LLM-as-Judge approach with weighted scoring across six criteria:

```
=====
EVALUATION SUMMARY
=====
Total Test Cases: 3
Passed: 3
Failed: 0
Pass Rate: 100.0%

*** AVERAGE SCORE: 3.63/5.00 ***
=====

CRITERION BREAKDOWN (averaged across all tests):
-----
accuracy      : 2.33/5.00 (weight: 25%)
completeness  : 5.00/5.00 (weight: 20%)
relevance     : 3.00/5.00 (weight: 20%)
tool_usage    : 3.00/5.00 (weight: 15%)
structure     : 5.00/5.00 (weight: 10%)
citations     : 5.00/5.00 (weight: 10%)

Results saved to: app/eval/results.csv

=====
EVALUATION COMPLETE
=====
```

Figure 9: LLM-as-Judge Evaluation Output

4.1 Evaluation Criteria and Weights

Table 7: LLM Judge Scoring Criteria

Criterion	Weight	Description
Accuracy	25%	Factual correctness of information provided
Completeness	20%	All required fields populated in TripPlan
Relevance	20%	Response addresses user's specific query
Tool Usage	15%	Appropriate tools called for the request
Structure	10%	Valid JSON conforming to Pydantic schema
Citations	10%	Sources provided for factual claims

4.2 Evaluation Results

Three test cases were evaluated:

1. Paris trip (June 2026) with BankGold card
2. Tokyo trip (March 2026) with BankPremium card
3. Barcelona trip (August 2026) with BankRewards card

Table 8: LLM Judge Scores by Criterion

Criterion	Average Score	Weight
Accuracy	2.33/5.00	25%
Completeness	5.00/5.00	20%
Relevance	3.00/5.00	20%
Tool Usage	3.00/5.00	15%
Structure	5.00/5.00	10%
Citations	5.00/5.00	10%
Overall	3.63/5.00	100%

Table 9: Evaluation Summary

Metric	Value
Total Test Cases	3
Passed	3
Failed	0
Pass Rate	100%
Average Score	3.63/5.00

5 RAG Implementation

5.1 Knowledge Base

The RAG system stores travel-related knowledge in Cosmos DB with vector embeddings:

```

13
14 class WeatherTools:
15     """Weather tools for the Travel Concierge Agent."""
16
17     @kernel_function(name="get_weather", description="Get weather forecast from Open-Meteo API for given coordinates")
18     def get_weather(self, lat: float, lon: float) -> dict:
19         """
20         Get weather forecast for given coordinates using Open-Meteo API.
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS AZURE

(.venv) PS C:\Training\Udacity\Microsoft_Foundry\Projects\Project_3> python app/scripts/test_rag.py

```

--- Query: How much travel insurance does BankPlatinum provide? ---
[OK] Query embedding generated (dim=1536)
[OK] VectorDistance query executed

Results with similarity scores:
-----

[1] Title: Travel Insurance Coverage
    VectorDistance: 0.746426
    Similarity Score: 0.2536 (25.4%)
    Content: Banking International cards include travel insurance when travel is purchased with the card. Coverag...

[2] Title: BankPlatinum Card Benefits
    VectorDistance: 0.638599
    Similarity Score: 0.3614 (36.1%)
    Content: BankPlatinum Card provides 5x points on dining worldwide, 4x points on travel and hotels, no foreign...

[3] Title: Airport Lounge Access Rules
    VectorDistance: 0.573624
    Similarity Score: 0.4264 (42.6%)
    Content: Banking International premium cardholders can access partner lounges worldwide. BankGold: 4 complime...

--- Query: What are the airport lounge access rules? ---
[OK] Query embedding generated (dim=1536)
[OK] VectorDistance query executed

```

Figure 10: RAG Retrieval with Vector Similarity Scores

Table 10: Knowledge Base Content

Category	Content
Card Benefits	BankGold: 4x dining, 3x travel, no FX fees
Card Benefits	BankPremium: 5x dining, 4x travel, lounge access
Card Benefits	BankRewards: 3x on all purchases, 1.5% cash-back
Lounge Rules	Priority Pass access requirements
Travel Policies	Insurance coverage details
MCC Benefits	Category-specific reward multipliers

5.2 Embedding Configuration

```

# Embedding generation using Azure OpenAI
embedding_service = AzureTextEmbedding(
    deployment_name="text-embedding-3-small",
    endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    api_version="2024-02-15-preview"
)

```

```
# Generate embeddings for knowledge snippets
embeddings = await embedding_service.generate_embeddings([text])
```

5.3 RAG Ingestion Results

```
[START] Starting knowledge base ingestion...
[OK] Embedding service initialized
[OK] Connected to Cosmos DB: ragdb/snippets
[OK] Upserted snippet: bankgold-benefits
[OK] Upserted snippet: bankpremium-benefits
[OK] Upserted snippet: bankrewards-benefits
[OK] Upserted snippet: lounge-rules
[OK] Upserted snippet: travel-insurance
[OK] Upserted snippet: dining-mcc
[OK] Upserted snippet: hotel-mcc
[OK] Ingested 7 documents
```

6 Key Implementation Details

6.1 8-Phase State Machine

The agent follows a structured workflow through 8 phases:

```
class AgentPhase(Enum):
    """Agent workflow phases"""
    Init = "Init"
    ClarifyRequirements = "ClarifyRequirements"
    PlanTools = "PlanTools"
    ExecuteTools = "ExecuteTools"
    AnalyzeResults = "AnalyzeResults"
    ResolveIssues = "ResolveIssues"
    ProduceStructuredOutput = "ProduceStructuredOutput"
    Done = "Done"
```

6.2 Tool Integration

Five tools are registered as Semantic Kernel plugins:

```
# Register tool plugins
kernel.add_plugin(WeatherTools(), plugin_name="WeatherTools")
kernel.add_plugin(FxTools(), plugin_name="FxTools")
kernel.add_plugin(SearchTools(), plugin_name="SearchTools")
kernel.add_plugin(CardTools(), plugin_name="CardTools")
kernel.add_plugin(KnowledgeTools(), plugin_name="KnowledgeTools")
```

6.3 Pydantic Output Validation

The TripPlan output is validated using Pydantic:

```
class TripPlan(BaseModel):
    """Validated trip plan output"""
    destination: str
    dates: TravelDates
    weather: WeatherInfo
    currency: CurrencyInfo
    recommendations: List[Recommendation]
    card_recommendation: CardRecommendation
    citations: List[Citation]
```

6.4 Bing Grounding Integration

Web search uses Azure AI Foundry Agent with Bing tool:

```
# Create AI Foundry Agent client
project_client = AIProjectClient(
    credential=DefaultAzureCredential(),
    endpoint=os.getenv("PROJECT_ENDPOINT")
)

# Execute search via agent
thread = project_client.agents.create_thread_and_process_run(
    assistant_id=os.getenv("AGENT_ID"),
    thread={"messages": [{"role": "user", "content": query}]}
)
```

7 Conclusions

This project successfully implements an AI Travel Concierge Agent using Semantic Kernel with Azure AI services. Key achievements include:

1. **Agentic Architecture:** 8-phase state machine with proper workflow management
2. **Tool Integration:** Five tools (Weather, Currency, Search, Card, Knowledge) as SK plugins
3. **RAG Implementation:** Vector embeddings stored in Cosmos DB with similarity search
4. **Bing Grounding:** Real-time web search via Azure AI Foundry Agent
5. **Structured Output:** Pydantic-validated TripPlan JSON schema
6. **LLM Evaluation:** 3.63/5.00 average score with 100% pass rate
7. **Comprehensive Testing:** 76 unit tests + 8/8 system checks passing

7.1 Technology Stack Summary

Component	Implementation
Orchestration	Semantic Kernel 1.36.1
LLM	Azure OpenAI gpt-4o, gpt-4o-mini
Embeddings	text-embedding-3-small (1536 dimensions)
Vector Store	Azure Cosmos DB (serverless)
Web Search	Bing Grounding via AI Foundry Agent
Weather API	Open-Meteo (free, no auth)
Currency API	Frankfurter (free, no auth)
Validation	Pydantic 2.11.7
Testing	pytest with 76 tests

7.2 Repository

All code is available at: <https://github.com/rodolfoermacontreras/-Udacity-Foundry-Project3>

End of Submission Report